

Modul 4

(LINKED LIST II)

DESKRIPSI TEMA

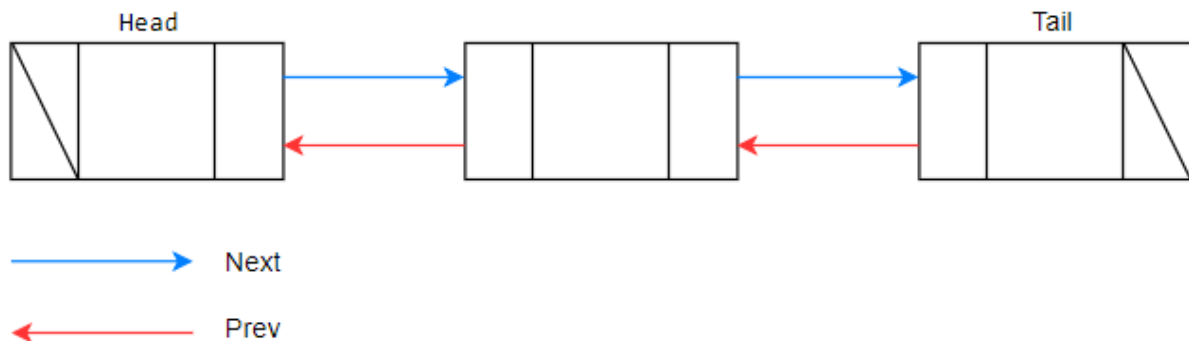
Double Linked List adalah struktur data yang terhubung secara dinamis dan memiliki minimal 3 jenis variabel. 2 variabel digunakan sebagai pointer yang mereferensikan node sebelum dan node sesudah di dalam suatu linked list dan space sisanya digunakan untuk menyimpan data dalam suatu node.

Kelebihan menggunakan Double Linked List yaitu dapat men-*traverse* node lain secara dinamis, yakni dapat dilakukan dari head (hingga ke tail) atau dari tail (hingga ke head).

Tahapan dalam pembuatan Double Linked List tidak jauh berbeda dari single linked list. Berikut hal yang harus diperhatikan dalam tahapan pembuatan Double Linked List:

1. Deklarasi 2 tipe data abstrak pointer, untuk menandai node sebelum dan sesudah.
2. Deklarasi pointer penunjuk kepala
3. Membentuk simpul pertama dan menunjuknya menjadi kepala
4. Membentuk simpul baru dan mengaitnya secara 2 arah ke simpul sebelumnya/sesudahnya.

Visualisasi gambar Double Linked List adalah sebagai berikut :

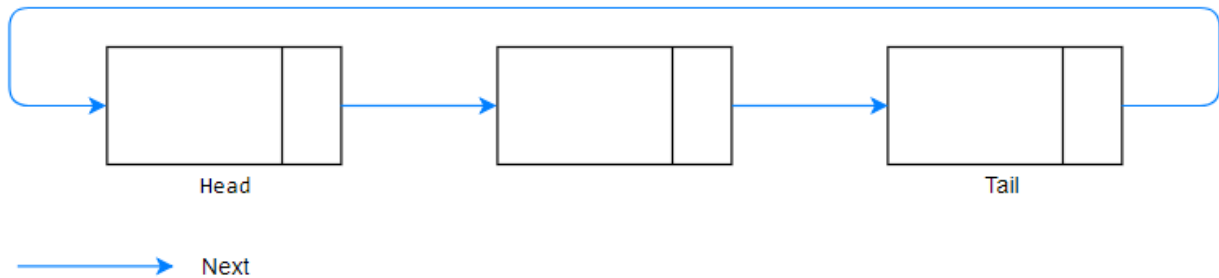


Circular Linked List adalah struktur data yang terhubung secara dinamis dan pointer next pada node akhir (tail) dalam linked list menunjuk ke node awal linked list (head).

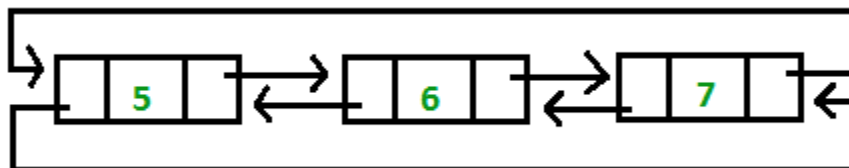
Adapun kelebihan dari penggunaan struktur data Circular Linked List seperti dapat mentraverse semua node mulai dari node manapun dan dapat dengan mudah menyelesaikan suatu problem yang menuntut penggunaan data sirkular.

Tahapan dalam pembuatan Circular Linked List juga tidak jauh berbeda dari single linked list. Hanya ada 1 langkah tambahan seperti mengaitkan pointer next di simpul terakhir (tail) linked list ke node

pertama (head). Visualisasi gambar Circular Linked List dengan menggunakan single linked list adalah sebagai berikut:



Circular Linked List tidak hanya dapat direpresentasikan dengan menggunakan Single Linked List saja, namun juga dapat direpresentasikan dengan menggunakan Double Linked List. Visualisasi gambar Circular Linked List dengan menggunakan double linked list adalah sebagai berikut :



Sumber: <https://www.geeksforgeeks.org/doubly-circular-linked-list-set-1-introduction-and-insertion/>

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa dapat memahami dan menerapkan konsep linked list (C₃).

PENUNJANG PRAKTIKUM

1. Software CodeBlocks

LANGKAH-LANGKAH PRAKTIKUM

Tutorial 1 – Contoh Implementasi

1. Double Linked List
 - a. Deklarasi Double Linked List secara jelas dicontohkan sebagai berikut:

```
struct tnode{
    int data;
    struct tnode *next, *prev;
};
```

- b. Deklarasi Pointer:

```
struct tnode *head, *node,*tail,*curr;
```

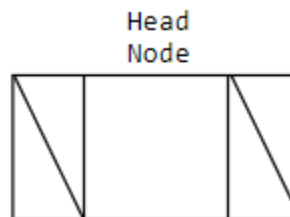
- c. Menyiapkan alamat memori untuk pointer node:

```
node = (struct tnode*) malloc (sizeof(struct tnode));
node->next = NULL;
node->prev = NULL;
```

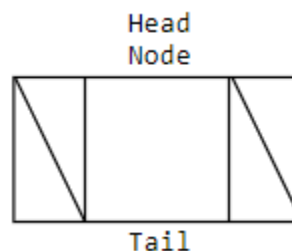
- d. Mengaitkan node pertama ke dalam linked list

```
if(head==NULL){
    head = node; //(1)
    tail = node; //(2)
}
```

- (1) Jika node yang baru dibuat merupakan node pertama, maka dihubungkan dengan head terlebih dahulu.



- (2) Karena dia merupakan node pertama (node satu-satunya) maka dia juga bertindak sebagai node terakhir.

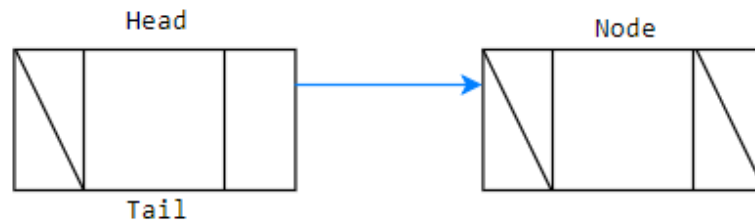


Langkahi step berikutnya jika node yang dibuat merupakan node pertama.

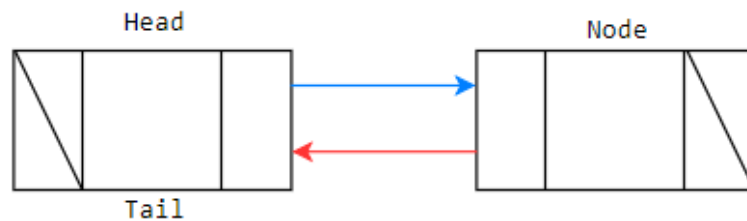
- e. Mengaitkan node lanjutan ke dalam linked list (jika node yang dibuat bukan node pertama)

```
else{
    tail->next=node; //(1)
    node->prev=tail; //(2)
    tail = node; //(3)
}
```

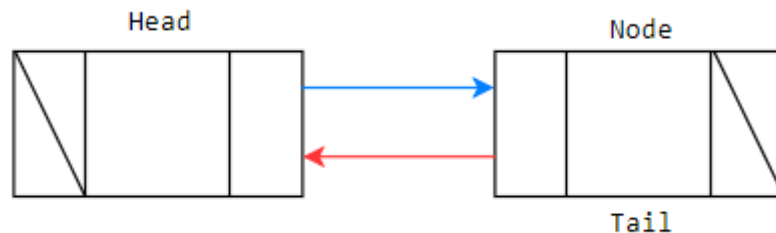
- (1) Karena node yang dibuat merupakan node ke-2 dan seterusnya, maka node tersebut harus dihubungkan dengan tail.



- (2) Pointer prev pada node juga harus di hubungkan ke pointer tail sebelum tail berubah menjadi node.



- (3) Setelah next dan prev sudah dihubungkan, node yang baru dibuat harus di set sebagai tail.



2. Circular Linked List – Single

Deklarasi Circular Linked List secara jelas, tidak ada beda dengan Single Linked List karena perbedaannya terdapat ketika mengimplementasikan seperti penambahan data ke linked list dan menghapus suatu simpul. Deklarasi CLL dicontohkan sebagai berikut :

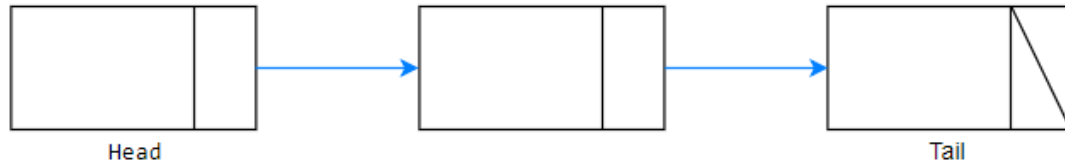
- a. Deklarasi struct:

```
struct tnode{
    int data;
    struct tnode *next;
};
```

- b. Deklarasi pointer

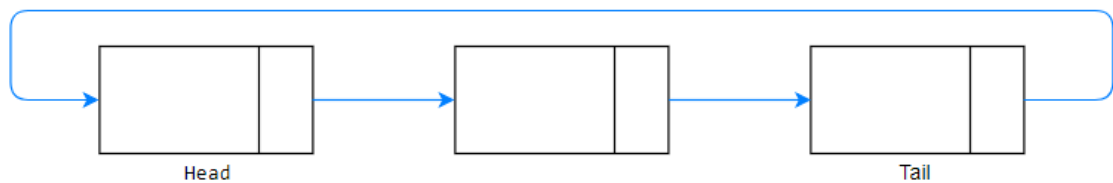
```
struct tnode *head, *node, *tail, *curr;
```

- c. Penyiapan alamat pada memori dan penghubungan node baru sama seperti single linked list, namun node terakhir di set sebagai tail



d. Sambungan tail ke head

```
else{
    tail->next=node; //sama seperti single linked list
    tail=node; //sama seperti single linked list
    tail->next=head;
}
```



Tutorial 2 – Let's Make it

Salinlah dan pahami potongan kode dibawah ini:

```
#include <stdio.h>
#include <malloc.h>

struct tnode {
    int data;
    struct tnode *next, *prev;
};
```

```
int main(int argc, char const *argv[]) {
    int bil, n, inp=1;
    struct tnode *head, *node, *tail, *curr;

    head = NULL;

    printf("Masukkan jumlah data: "); scanf("%d", &n);

    while(n > 0) {
        printf("Masukkan bil = "); scanf("%d", &bil);

        node = (struct tnode*) malloc(sizeof(struct tnode));
        node->next = NULL;
        node->prev = NULL;
        node->data = bil;

        if(head == NULL) {
            head = node;
            tail = node;
        }
        else {
            tail->next = node;
            node->prev = tail;
            tail = node;
        }
        n--;
    }
}
```

```
curr = head;

while(1) {
    system("cls");
    printf("%d\n", curr->data);
    printf("1. Maju\n");
    printf("2. Mundur\n");
    printf("0. Keluar\n");
    printf("Pilihan: "); scanf("%d", &inp);

    if(inp == 1) {
        if(curr->next == NULL) {
            printf("Tidak dapat maju karena node selanjutnya kosong\n");
            getch();
        }
        else curr = curr->next;
    }
    else if(inp == 2) {
        if(curr->prev == NULL) {
            printf("Tidak dapat mundur karena node sebelumnya kosong\n");
            getch();
        }
        else curr=curr->prev;
    }
    else if(!inp) break;
};

return 0;
}
```

Jawablah pertanyaan berikut! Simpanlah dalam sebuah file .txt dengan format nama **NIM_Nama_T1.txt**!

1. Variasi linked list apa yang digunakan dalam potongan kode diatas (single/double/circular)? Jelaskan!

2. Penambahan simpul dalam kodingan tersebut merupakan tambah belakang atau tambah depan?
Jelaskan!

Salinlah dan pahami potongan kode dibawah ini!

```
#include <stdio.h>
#include <malloc.h>

struct tnode {
    int data;
    struct tnode *next;
};

int main(int argc, char const *argv[]) {
    int bil, n, inp=1;
    struct tnode *head, *node, *tail, *curr;

    head = NULL;

    printf("Masukkan jumlah data: "); scanf("%d", &n);

    while(n > 0) {
        printf("Masukkan bil ke-%d = ", n); scanf("%d", &bil);

        node = (struct tnode*) malloc(sizeof(struct tnode));
        node->data = bil;

        if(head == NULL) {
            head = node;
            tail = node;
            tail->next = NULL;
            head->next = NULL;
        }
        else {
            tail->next = node;
            tail = node;
            tail->next = head;
        }
        n--;
    }
}
```

```

curr = head;

while(inp) {
    system("cls");
    printf("%d\n", curr->data);
    printf("1. Maju\n");
    printf("0. Keluar\n");
    printf("Pilihan: "); scanf("%d", &inp);

    if(inp == 1) {
        curr = curr->next;
    }
    else if(inp > 1 || inp < 0) {
        printf("Maaf input salah !\n");
        getch();
    }
};

return 0;
}

```

Jawablah pertanyaan berikut! Simpanlah dalam sebuah file .txt dengan format nama **NIM_Nama_T2.txt**!

1. Variasi linked list apa yang digunakan dalam potongan kode diatas (single/double/circular)? Jelaskan!
2. Penambahan simpul dalam kodingan tersebut merupakan tambah belakang atau tambah depan? Jelaskan!

Challenge

Format nama file: **NIM_challenge.c**

Challenge pada week ini merupakan pengembangan terhadap apa yang telah kalian kerjakan di Week 04, sehingga kalian akan memerlukan tugas Week 04.

Dalam tugas ini kalian harus mengubah Single Linked List yang ada pada Week 04 menjadi Double Linked List dan menambahkan opsi sebagai berikut:

```

=====
Menu peminjaman ruangan ver 0.2
=====
1. Pinjam ruangan
2. List Ruangan yang dipinjam
3. Hapus list ruangan yang dipinjam(Clear All)
4. Menyetujui ruangan
5. Batalkan peminjaman ruangan
6. Exit
Input :

```


Pada menu pinjam ruangan, bagian yang merupakan bonus/optional pada Week 04 yaitu tidak bisa memasukkan ruangan yang sama untuk week ini harus disertakan!

Menu penyetujuan ruangan:

```
=====
Penyetujuan Ruangan
=====
Nomor    : 1 / 3
Nama     : Justin Susanto
Nim      : 00000013613
Prodi    : Informatika
Angkatan : 2016
Ruangan  : B501
Status   : Unapproved
Menu :
1. Next
2. Prev
3. Approve
4. Exit
Pilihan :
```

Jika ditekan next:

```
=====
Penyetujuan Ruangan
=====
Nomor    : 2 / 3
Nama     : Mario Alexander
Nim      : 00000013728
Prodi    : Informatika
Angkatan : 2016
Ruangan  : B503
Status   : Unapproved
Menu :
1. Next
2. Prev
3. Approve
4. Exit
Pilihan :
```

Jika di approve:

```
=====
Penyetujuan Ruangan
=====
Nomor      : 2 / 3
Nama       : Mario Alexander
Nim        : 00000013728
Prodi      : Informatika
Angkatan   : 2016
Ruangan    : B503
Status     : Approved
Menu :
1. Next
2. Prev
3. Approve
4. Exit
Pilihan : 3
Already Approved
Press any key to continue . . .
```

Untuk keterangan selanjutnya anda bisa melihat contoh exe yang disertakan untuk week kali ini.

(hint: gunakanlah konsep yang telah kalian pelajari pada week ini yaitu double linked list dan circular linked list untuk challenge kali ini).

Untuk menu Pembatalan:

```
=====
Penyetujuan Ruangan
=====
Nomor      : 2 / 3
Nama       : Mario Alexander
Nim        : 00000013728
Prodi      : Informatika
Angkatan   : 2016
Ruangan    : B503
Status     : Approved
Menu :
1. Next
2. Prev
3. Cancel
4. Exit
Pilihan :
```

Jika dibatalkan:

```
=====
Penyetujuan Ruangan
=====
Nomor   : 1 / 2
Nama    : Justin Susanto
Nim     : 00000013613
Prodi   : Informatika
Angkatan : 2016
Ruangan : B501
Status  : Unapproved
Menu :
1. Next
2. Prev
3. Cancel
4. Exit
Pilihan :
```

Program harus bisa melakukan pembatalan di semua rantai yang tersambung pada list (list ditengah, di head, maupun di tail)

REFERENSI